

26ICS Lab0 实验报告

汪子斐 24300700002

【实验目的】

本实验的目标是掌握 Git 的基本提交过程，理解工作区、暂存区和仓库之间的关系，并通过真实分支冲突练习分支创建、切换、合并和冲突处理。

【实验问题】

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我此前很少系统使用 Git 工作流完成多人协同开发。本次实验由我在个人仓库中独立完成，因此没有多人同时修改仓库的情况。通过分支冲突练习，我理解到多人协作时应按功能建立分支，用清晰的提交信息记录修改，再通过合并或 Pull Request 集成代码。若两个人修改同一位置，Git 会保留两侧内容并要求开发者根据实际意图处理冲突。

2. Git 为什么要设计“暂存-提交”两个步骤？

工作区中的修改可能属于不同任务。暂存区允许从当前所有修改中挑选下一次提交需要包含的内容，并在提交前使用 `git diff --staged` 检查。这样可以把一次提交限制在一个明确目的内，避免将代码、调试内容和文档混在一起。小而完整的提交更容易审查、回退，也方便以后使用 `cherry-pick` 复用。

3. `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

`git branch` 默认只显示本地分支，并用星号标记当前分支。`git branch -a` 中的 `-a` 表示 `-all`，它除了显示本地分支，还显示本地已经获取到的远程跟踪分支，例如 `remotes/origin/main`。远程跟踪分支是本地保存的远程状态，需要执行 `git fetch` 才会更新。

本仓库运行结果中，本地分支为 `main` 和 `feature`；使用 `git branch -a` 时还可以看到 `remotes/origin/HEAD` 指向 `origin/main`，以及 `remotes/origin/main`。

【阅读与理解】

1. Commit Message 规范概括

提交信息用于说明一次修改的目的。文章介绍的规范包含标题、可选正文和页脚：标题

概括修改类型与范围，正文解释原因，页脚可补充不兼容变化或关联问题。常见类型有 **feat**（新功能）、**fix**（修复问题）和 **docs**（文档修改）。这些是团队约定，并非 **Git** 强制要求的格式。

清楚的说明便于浏览历史、定位问题和整理发布日志。提交信息对应一次修改，发布日志则概括一个版本面向使用者的变化。本实验的提交分别记录了初始输出、两个分支的修改和冲突处理。如果在合并提交中补充“保留双方输出并按顺序打印”，就能进一步解释最终结果。

2. 语义化版本概括

语义化版本采用“主版本号.次版本号.修订号”。在公共接口已明确的前提下，不兼容修改递增主版本号，兼容的功能新增递增次版本号，兼容的问题修复递增修订号。例如，从 1.2.3 出发，兼容修复可发布为 1.2.4，兼容新增可发布为 1.3.0，不兼容修改则发布为 2.0.0。已经发布的版本不应被替换，后续修改应使用新版本号。

版本号传达的是兼容性，而不是代码修改量。本实验从一行输出变成两行，若供人阅读，影响可能较小；若其他程序依赖固定单行格式，就可能破坏约定。因此，应先明确对外接口，再判断版本变化。

3. 为什么要学习 Git

Git 使实验过程能够追踪和比较，通过保存每次有意义的修改，使我能够比较版本、回退错误并在独立分支上试验。本次 **main** 和 **feature** 修改了同一条语句，合并时出现冲突。我保留两条 **printf**、删除冲突标记并提交，运行后得到两行预期输出。这让我认识到，解决冲突需要判断最终行为，不能机械地选择某一方。因此，即使项目只有一个开发者，**Git** 历史也能帮助以后理解代码是如何形成的。多人协作时，分支和合并为并行开发提供明确边界，冲突标记则指出必须人工判断的位置。规范的提交信息还能解释修改原因。对个人学习，**Git** 便于保留阶段成果、查找错误；对多人合作，它支持交换提交、审查差异和整合修改。

【实验步骤】

1. 完成 main.c 文件中的 TODO 部分并进行一次 commit。

进入 **main.c**，将输出改为 **This is a very valuable sentence**，通过 **git add main.c** 保存输出

修改，创建提交 eb9d60，提交信息为 complete main output task。

2. 学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交。

git switch -c feature 新建分支，git branch 确认当前处于 feature 分支。进入 main.c 把输出内容改为 This is a good sentence in feature branch，对应提交 4dcf50a。git switch main 换回 main 分支，进入 main.c 将输出改为分支改为 This is a better sentence than before，对应提交 7a489bf。

3. 随后将 feature 分支 merge 到 main 分支，并处理发生的合并冲突

git branch 确认当前处于 main 分支，执行 git merge feature，终端报告 main.c 出现内容冲突，并提示需要先修复冲突再提交。这个结果说明两个分支确实形成了题目要求的冲突。

```
(base) 11debijibendiannao:Lab_ics260 a11$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

图 1 执行 git merge feature 后出现内容冲突

VS Code 在 main.c 中显示了两侧修改。HEAD 一侧是 main 分支的句子，feature 一侧是传入分支的句子；中间的等号用于分隔两组内容。

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      <<<<<< HEAD (当前更改)
7      printf("This is a better sentence than before.\n");
8      =====
9      printf("This is a good sentence in feature branch.\n");
10     >>>>>> feature (传入的更改)
11 }
```

图 2 main.c 中的冲突标记和两个分支内容

通过删除冲突标记并保留两条 printf 语句。之后暂存 main.c 并提交，生成合并提交 f4c79ff，提交信息为 resolve feature message conflict。提交图显示 main 和 feature 从同一节点分叉，再由合并提交连接。

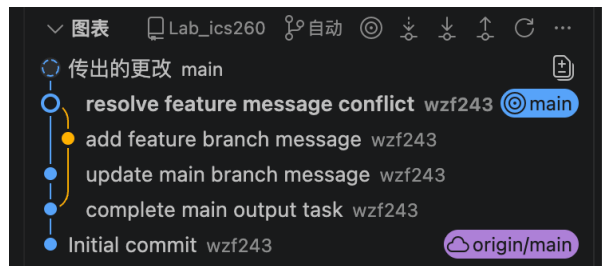


图 3 分支分叉与冲突解决后的合并

使用 Makefile 编译 main.c。运行 ./main 后依次输出 main 和 feature 的两条句子。

```
(base) 11debijibendiannao:Lab_ics260 a11$ ./main
This is a better sentence than before.
This is a good sentence in feature branch.
```

图 4 合并后程序的实际运行结果

```
* f4c79ff resolve feature message conflict
|\
| * 4dcf50a add feature branch message
* | 7a489bf update main branch message
|/
* ebf9d60 complete main output task
* 7f7dd8d Initial commit
```

图 5 提交分支记录

【实验总结与建议】

这次操作让我理解到，合并冲突不是 Git 发生故障，而是 Git 无法替开发者判断哪一侧内容更符合需求。解决冲突前应先阅读 HEAD 和传入分支的修改，再决定保留、删除或组合内容。完成编辑后，还需要重新编译运行，并查看提交图确认合并结构正确。

建议在后续实验中继续保持一次提交只完成一件事，并在切换分支、合并和提交前运行 git status。实验说明还可以增加一次 git diff --staged 的练习，使暂存区的作用更直观。